

Opgave 3 – LAMP i Azure

I denne opgave skal I designe, deploye, sikre og teste en LAMP-baseret webapplikation i Azure.

I arbejder i tre faser: **1. Cloud-økonomi** → **2. Design** → **3. Deploy**

Cloud-økonomi – Cloud Survival Challenge

Bemærk!: I har **\$200 Azure-credit**.

Jeres mål er at få en fungerende LAMP-løsning til at køre **så længe som muligt** inden for jeres budget.

Undersøg og vurder:

- Hvilke Azure-ressourcer har I brug for?
- Hvilken region vil I bruge?
- Hvilken VM-størrelse vil I vælge?
- Hvad koster løsningen pr. dag og pr. måned?
- Hvor længe kan løsningen køre for \$200?

<https://azure.microsoft.com/da-dk/pricing/calculator/>

Hvilke designvalg påvirker prisen?

Sammenlign kort **On-premises vs. Azure** og forklar jeres valg.

Challenge: Kan I designe en løsning, der kan køre i **100 dage for \$200**?

Den skal stadig være **funktionel, sikker og testbar**.

Kan I forklare, hvorfor jeres løsning koster det, den gør

— og hvorfor I har valgt den?

Design en LAMP / LEMP-stack i Docker

I får udleveret en simpel LAMP/LEMP-applikation eller i kan kode en selv 😊.

LAMP/LEMP består af: **Linux-Apache/NGINX-MySQL/MariaDB-PHP**

<https://learn.microsoft.com/en-us/azure/virtual-machines/linux/tutorial-lemp-stack>

Design, hvordan applikationen skal køre i Docker.

Overvej:

- Hvilke containers har I brug for?
- Hvordan kommunikerer de? Hvilke porte skal være åbne?
- Hvor gemmes data?
- Hvordan håndteres credentials / Logins ?
- Hvordan kan løsningen senere flyttes til Azure? Brug evt. **Docker Compose**.

Design samtidig Azure-infrastrukturen:

- Resource Group
- <https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/>
- NSG (***An Azure Network Security Group (NSG) filters network traffic to and from Azure resources in a virtual network***)
 - Public IP – til test at det virker !
 - Linux VM med docker.

NB! Lav et **arkitekturdiagram**, der viser både applikation og infrastruktur først.

Deploy din LAMP/LEMP-stack i Azure

Deploy løsningen til egen Azure.

Azure-infrastrukturen skal oprettes med **Terraform (laC)**.

<https://learn.microsoft.com/en-us/training/paths/terraform-fundamentals/>

Som minimum skal Terraform oprette:

- Resource Group -VNet – Subnet - NSG - Public IP - NIC - Linux VM
- VM'en skal konfigureres automatisk med **Cloud-init**.
- Docker og LAMP/LEMP-applikationen skal starte automatisk.

Sikkerhed

Løsningen skal som minimum:

- kun eksponere nødvendige porte
- beskytte VM-administrationen
- beskytte databaseadgangen
- håndtere secrets forsvarligt
- forhindre direkte adgang til MariaDB-porten fra internettet

Test

Test at:

- webapplikationen virker
- Docker-containere kører
- databasen fungerer
- relevante porte er åbne
- databasen ikke er offentligt tilgængelig

Lav mindst 2 **Playwright E2E-tests** mod jeres Azure-deployment (IP-adresse).

Dokumentation

Jeres GitHub repository skal indeholde:

- Docker Compose + Terraform + Cloud-init + Playwright-tests
- README + arkitekturdiagram

README skal beskrive:

1. Hvordan løsningen oprettes
2. Hvordan den testes
3. Hvordan den opdateres
4. Hvordan den fjernes

NB! Løsningen skal kunne fjernes igen med: terraform destroy

Opgaven er færdig, når:

- ***LAMP/LEMP virker lokalt med Docker Compose***
- ***Azure-arkitekturen er designet***
- ***Terraform opretter infrastrukturen***
- ***VM'en konfigureres automatisk***
- ***LAMP/LEMP kører i Azure med WAN-IP***
- ***MariaDB-porten er ikke offentligt tilgængelig***
- ***Der er lavet mindst 3 Playwright-tests***
- ***Projektet ligger i egen GitHub***
- ***README og diagram er på plads***
- ***I kan forklare jeres cloud-økonomi***
- ***I kan forklare jeres sikkerhedsvalg***
- ***Løsningen kan fjernes med terraform destroy***